

Documentação do Interpretador de Template

- Documentação do Interpretador de Template
 - Introdução
 - Estrutura do Template
 - Delimitadores
 - Alinhamento do Label
 - Campos de Entrada
 - Funcionamento do Método `parseTemplate`
 - Exemplo de Template
 - Interpretação do Exemplo
 - Conclusão
 - Referências

Introdução

Este documento fornece uma descrição detalhada da linguagem usada no método `parseTemplate`, que é um interpretador de templates. O método analisa um template de texto e extrai informações estruturadas a partir dele, organizando-as em um formato que pode ser utilizado posteriormente.

Estrutura do Template

O template é composto por linhas de texto, onde cada linha pode conter labels e campos de entrada. A sintaxe do template é definida pelos seguintes elementos:

Delimitadores

- `~`: Este caractere é usado para alternar entre a definição de um label e a entrada de um campo. Quando um `~` é encontrado, o interpretador alterna o estado de "label" para "valor" e vice-versa.
- `\`: Este caractere é utilizado para indicar o início de um campo de entrada. Após a barra invertida, o interpretador espera encontrar a quantidade de caracteres que o campo deve suportar, seguida pela definição do campo.
- `^B`: Este conjunto de caracteres indica o término da definição do tamanho do campo e o início do nome do campo. O nome do campo é extraído até o próximo espaço ou o final da linha.

Alinhamento do Label

Os labels podem ter um alinhamento definido com base nos caracteres `::`:

- `:label::`: O label é centralizado.

- **:label**: O label é alinhado à esquerda.
- **label**:: O label é alinhado à direita.

Campos de Entrada

Os campos de entrada são definidos após a barra invertida e podem incluir:

- Um número de caracteres **s** que define o comprimento máximo do campo.
- Um nome de campo que é extraído após o delimitador **^B**.

Funcionamento do Método **parseTemplate**

O método **parseTemplate** processa o template da seguinte forma:

1. **Divisão em Linhas**: O template é dividido em linhas com base no caractere de nova linha (**\n**).
2. **Iteração sobre as Linhas**: Cada linha é analisada individualmente. Linhas vazias são ignoradas.
3. **Processamento de Caracteres**:
 - O método percorre cada caractere da linha, alterando entre estados de label e valor conforme necessário.
 - Quando um **** é encontrado, ele verifica a quantidade de caracteres **s** que seguem, que determina o comprimento máximo do campo.
 - O nome do campo é extraído após o delimitador **^B**.
4. **Adição de Campos**: Quando um label e um campo são definidos, eles são adicionados a uma estrutura de dados (**currentLine**), que é então armazenada em um array (**this.fields**).
5. **Reset de Variáveis**: Após processar uma linha, as variáveis são redefinidas para garantir que os dados da próxima linha sejam processados corretamente.

Exemplo de Template

```
~ Nome:~ \ssssssssssssssssssssssssssss^B nomeCompleto
~ Idade: ~ \ss^B idadeUsuario
~ Cidade: ~ \ssssssssssssssss^B cidadeUsuario
```

Interpretação do Exemplo

- Linha 1: ~ Nome: ~ \ssssssssssssssssssssssssssss^B nomeCompleto
 - Define um campo de entrada para "Nome" onde o rótulo Nome é alinhado pela direita.
 - O campo deve ter um comprimento máximo de 30 caracteres.
 - O nome do campo será referenciado como nomeCompleto.

- Linha 2: ~ Idade:~ \ss^B idadeUsuario
 - Define um campo para "Idade" onde o rótulo Idade é alinhado pela direita.
 - O campo deve ter um comprimento máximo de 2 caracteres.
 - O nome do campo será referenciado como idadeUsuario.
- Linha 3: ~ Cidade ~ \ssssssssssssss^B cidadeUsuario
 - Define um campo para "Cidade" onde o rótulo Cidade é alinhado pela direita.
 - O campo deve ter um comprimento máximo de 15 caracteres.
 - O nome do campo será referenciado como cidadeUsuario.

Conclusão

O método `parseTemplate` é uma ferramenta poderosa para interpretar templates de texto, permitindo a definição de labels e campos de entrada de forma estruturada. A sintaxe é projetada para ser simples e intuitiva, facilitando a criação de templates personalizáveis. Esta documentação serve como um guia para entender e utilizar a linguagem do interpretador de templates.

Referências

- [código fonte](#)